UNIT 5 Application layer

The Application Layer is the topmost layer in the OSI Model and the TCP/IP Model, providing interfaces and protocols for users and applications to access network services. It directly interacts with software applications to facilitate communication and data exchange over a network.

Functions of the Application Layer:

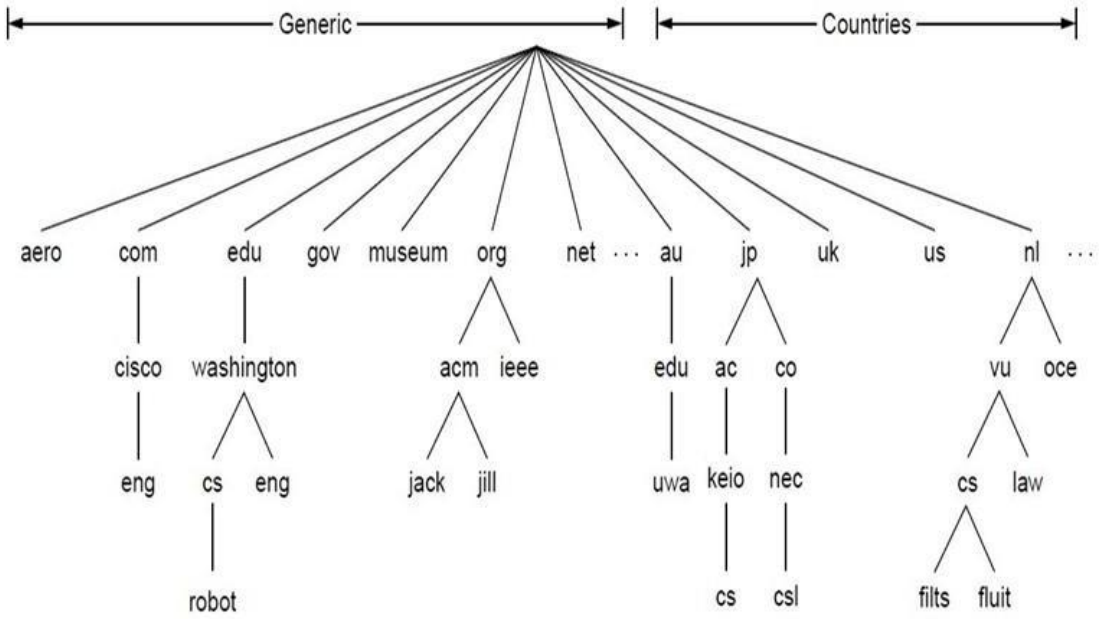
1. User Interface:
   - Provides a user-friendly interface for network interaction.
   - Example: Browsers, email clients, and file transfer tools.
2. Data Presentation:
   - Ensures that data is presented in a format understandable by applications.
   - Examples: Encryption, decryption, or data compression.
3. Protocol Implementation:
   - Supports application-specific communication protocols like HTTP, FTP, and SMTP.
4. Data Exchange:
   - Establishes, manages, and terminates connections between applications on different devices.
   - Ensures data integrity during transmission.

Domain name system:

The Domain Name System (DNS) is a hierarchical and decentralized naming system used to translate human-readable domain names (like www.example.com) into IP addresses (like 192.0.2.1) that computers use to identify and communicate with each other over the internet. It functions like a phonebook for the internet.

- DNS stands for Domain Name System.
- DNS is a directory service that provides a mapping between the name of a host on the network and its numerical address.
- DNS is required for the functioning of the internet.
- Each node in a tree has a domain name, and a full domain name is a sequence of symbols specified by dots
- .DNS is a service that translates the domain name into IP addresses. This
-

- allows the users of networks to utilize user-friendly names when looking for other hosts instead of remembering the IP addresses.
- For example, suppose the FTP site at EduSoft had an IP address of 132.147.165.50, most people would reach this site by specifying ftp.EduSoft.com. Therefore, the domain name is more reliable than IP address.



- **Domain Name Space** is the **organization** or **structure** of domain names.
- **Domain Name System** is the **system** that actually makes domain names work by resolving them to IP addresses.

Types of Domain Name System.:

- 1.Genericdomain:. It defines the registered hosts according to their generic behavior.
- Each node in a tree defines the domain name, which is an index to the DNS database.

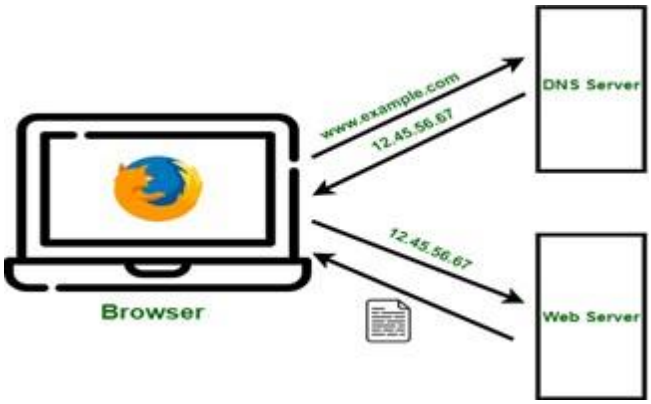
It uses three-character labels, and these labels describe the organization type. com:(commercial).edu(educational) .mil(military) .org(non profit organization) .net(similar to commercial) all these are generic domain.

2. Country domain:

The format of country domain is same as a generic domain, but it uses two-character country abbreviations (e.g., us for the United States) in place of three character organizational abbreviations.

.in (India) .us(united states) .uk

3. Inverse domain: The inverse domain is used for mapping an address to a name. When the server has received a request from the client, and the server contains the files of only authorized clients. To determine whether the client is on the authorized list or not, it sends a query to the DNS server and ask for mapping an address to the name.



How DNS Works (Step-by-Step):

1. **You Type a Website Address**:
   - You type **www.example.com** into your browser.
2. **Browser Checks Local Cache**:
   - Your computer checks its memory (cache) to see if it already knows the IP address for the website.
   - If found, it uses the IP address.
3. **Query to DNS Resolver**:
   - If not found, your computer asks a **DNS resolver** (usually provided by your ISP or a public service like Google DNS).
4. **Recursive Query**:

- The resolver asks a series of DNS servers to find the IP address:
    - **Root Server**: Knows where to find top-level domains (like .com, .org).
    - **TLD Server**: Knows where to find domain-specific servers (like example.com).
    - **Authoritative Server**: Knows the exact IP address for the domain.

5. **IP Address Returned**:
    - The resolver sends the IP address back to your browser.
6. **Website Loaded**:
    - The browser uses the IP address to connect to the website's server and load the page.

Domain namespace

Domain Namespace

The **Domain Namespace** is the hierarchical structure used by the Domain Name System (DNS) to organize and manage domain names. It defines the way domain names are structured, distributed, and resolved into IP addresses.

Structure of the Domain Namespace:

The domain namespace is organized as an **inverted tree**, starting from the **root node** at the top and branching out into various levels.

1. **Root Node (.)**:
    - The topmost level of the DNS hierarchy.
    - Represented as a single dot (.), but it is usually implicit in domain names.
    - Contains information about **Top-Level Domains (TLDs)**.
2. **Top-Level Domains (TLDs)**:
    - Directly below the root node.
    - Examples:
        - **Generic TLDs (gTLDs)**: .com, .org, .net
        - **Country Code TLDs (ccTLDs)**: .uk, .in, .jp
        - **Sponsored TLDs (sTLDs)**: .gov, .edu, .mil
3. **Second-Level Domains**:
    - Subdomains of the TLD.
    - Example: In example.com, example is the second-level domain under the .com TLD.
4. **Subdomains**:
    - Additional subdivisions of a second-level domain.
    - Example: In blog.example.com, blog is a subdomain of example.com.
5. **Hostnames**:
    - The final level of the hierarchy, representing individual devices or services.
    - Example: In www.example.com, www is a hostname.

Key Characteristics:

- **Hierarchical**:
    - Domains are arranged in a tree-like structure for scalability and efficient resolution.

- **Distributed**:
  - DNS is a distributed system; no single server contains the entire namespace.
- **Unique**:
  - Each domain name is unique within the namespace.
- **Human-Friendly**:
  - Names like example.com are easier for humans to remember compared to IP addresses.

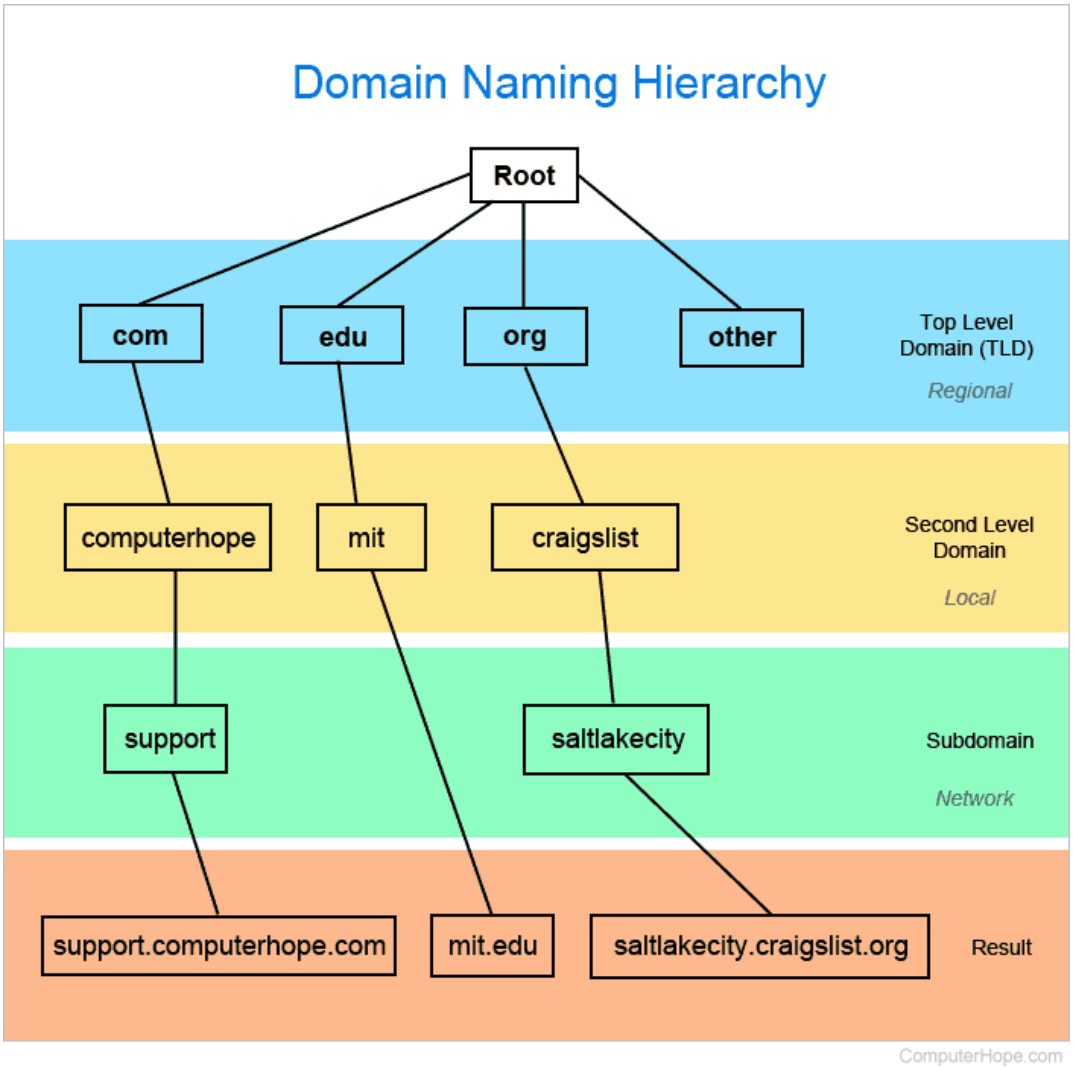
Example of the Domain Namespace:

- Root Node: .
  - TLD: .com
    - Second-Level Domain: example.com
      - Subdomain: blog.example.com
      - Hostname: www.example.com

Purpose:

- It allows humans to use readable names (e.g., `example.com`) to access Internet resources rather than IP addresses (e.g., `192.0.2.1`).

- It enables the decentralization and organization of domain registrations and management.



Resource record

DNS Resource Records

A **Resource Record (RR)** in DNS is like a data entry that helps the system manage domain names.They store information about a domain name and are used to answer DNS queries. A **Resource Record (RR)** in DNS is a basic unit of information that provides specific details about a domain name, such as its IP address, mail server, or other configurations. It helps the DNS system translate domain names into actions, like connecting to a website or routing emails.

Structure of a Resource Record:

A resource record has the following fields:

1. **Name**: The domain name for which the record applies (e.g., example.com).
2. **Type**: Specifies the type of information in the record (e.g., A, AAAA, MX).
3. **Class**: Almost always IN (Internet).
4. **TTL (Time to Live)**: The duration (in seconds) the record can be cached by resolvers.
5. **Data**: The value associated with the record (e.g., an IP address or a mail server).

Common Types of DNS Resource Records:

1. **A Record (Address Record)**:
    - Maps a domain name to an IPv4 address.
    - Example: example.com -> 192.0.2.1
2. **AAAA Record (IPv6 Address Record)**:
    - Maps a domain name to an IPv6 address.
    - Example: example.com -> 2001:0db8::1
3. **MX Record (Mail Exchange Record)**:
    - Specifies mail servers for a domain.
    - Example: example.com -> mail.example.com, priority=10
4. **CNAME Record (Canonical Name Record)**:
    - An alias for another domain name.
    - Example: www.example.com -> example.com
5. **NS Record (Name Server Record)**:
    - Specifies the authoritative name servers for a domain.
    - Example: example.com -> ns1.example.com
6. **TXT Record (Text Record)**:
    - Stores arbitrary text, often used for domain verification or security.

- Example: example.com -> "v=spf1 include:example.net ~all"

7. **PTR Record (Pointer Record)**:
   - Used for reverse DNS lookups, mapping an IP address to a domain name.
   - Example: 192.0.2.1 -> example.com
8. **SRV Record (Service Record)**:
   - Specifies services available for a domain.
   - Example: _sip._tcp.example.com -> sipserver.example.com
9. **SOA Record (Start of Authority)**:
   - Contains administrative information about the domain, such as the primary name server and contact email.
   - Example: example.com -> ns1.example.com, admin@example.com
10. **DNSKEY Record**:
   - Part of DNSSEC, contains public keys for secure DNS responses.

How Resource Records Work:

When a DNS query is made, the authoritative name server returns the relevant resource records to the resolver. For example:

- A browser requests the IP address for www.example.com.
- The name server responds with an A record containing the IPv4 address.

What is a Name Server?
A **Name Server** is a server that is responsible for translating human-readable domain names (such as `www.example.com`) into machine-readable IP addresses (such as `192.168.1.1`). This process is known as **name resolution**, and it is a fundamental part of the Domain Name System (DNS).

How Name Servers Work: Step-by-Step
1. User Request

When you type a domain name (e.g., `www.example.com`) into your browser and press Enter:

- The browser sends a request to the **recursive resolver** (a local DNS server, often managed by your Internet Service Provider or a public DNS service like Google DNS or Cloudflare).

2. Recursive Query

The recursive resolver checks whether it has the domain information cached locally. If not, it begins a query process by contacting other DNS servers in a hierarchical manner:

- **Root Name Server**: The resolver queries the root name server (e.g., `.`) to determine which Top-Level Domain (TLD) name server to contact (e.g., `.com` or `.org`).
- **TLD Name Server**: The resolver queries the TLD name server (e.g., `.com`) to find the authoritative name server for the specific domain (e.g., `example.com`).
- **Authoritative Name Server**: Finally, the resolver contacts the authoritative name server for `example.com`, which provides the requested IP address (e.g., `93.184.216.34`).

3. Response and Access

The resolver returns the IP address to the browser, which then uses it to establish a connection with the web server hosting the domain. The user sees the website.

Name Server is a system or server that holds and answers queries about domain names, while a **Resource Record** is the actual data (like an IP address or mail server) that the Name Server provides in response to a query.

SNMP (Simple Network Management Protocol) is an Internet Standard protocol used for managing and monitoring network-connected devices in IP networks. It is widely used for network management tasks like detecting network faults, monitoring network health, and even configuring remote devices.

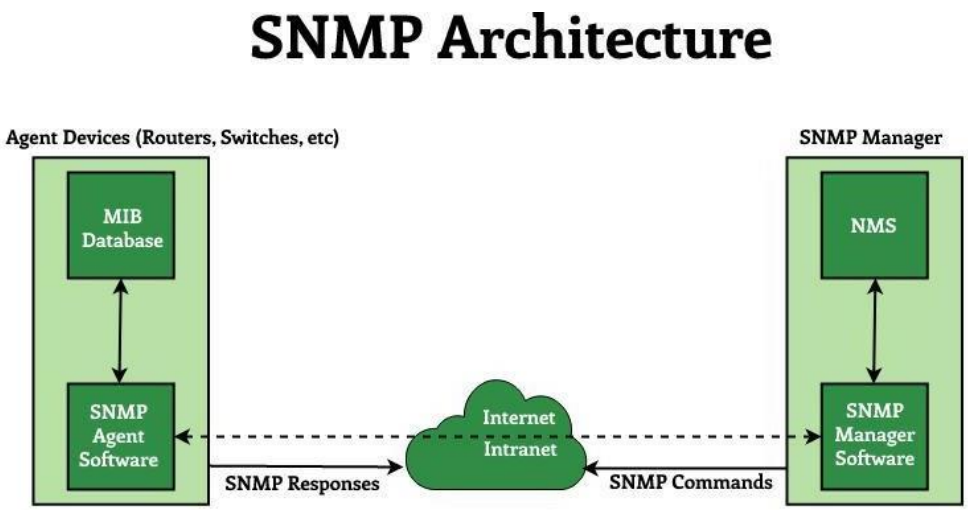
Key Components of SNMP:

1. **SNMP Manager** (Network Management Station - NMS):
    - The **SNMP Manager** is the centralized system responsible for managing the network.
    - It communicates with **SNMP Agents** running on devices to retrieve information or send configuration commands.
    - It can manage multiple devices on the network at once.
    - The SNMP Manager acts as the "client" in the client-server architecture, making requests to agents and interpreting responses.
2. **SNMP Agent**:
    - The **SNMP Agent** is a software module installed on network devices, such as routers, switches, printers, servers, and other network-connected devices.
    - The agent collects information about the device's status, performance, and health and stores this information in a **Management Information Base (MIB)**.
    - It responds to requests made by the SNMP Manager by sending back information about the device or performing requested actions like reconfiguring the device.
3. **Management Information Base (MIB)**:
    - The **MIB** is a collection of data that represents the various components of a managed device. It is essentially a hierarchical database of objects and variables that describe the device's attributes and its performance.

- Each object in the MIB is an identifier for a variable, and it holds information about the device's performance, such as network traffic, memory usage, CPU load, or error counts.
- **MIB Objects** are organized in a tree structure, and each object is identified by a unique object identifier (OID), which allows the SNMP Manager to request specific information from the SNMP Agent.

The MIB includes categories like:

- **System**: Information about the device itself (e.g., name, description, location).
- **Interface**: Information about the device's network interfaces (e.g., number of packets received, sent, errors).
- **TCP/UDP**: Information about the TCP and UDP protocols, such as connection stats.
- **ICMP**: Information about the Internet Control Message Protocol (ICMP) like echo requests and replies.
- **IP**: Information about the IP protocol, such as packet forwarding and routing.
- **EGP**: Exterior Gateway Protocol stats (for routing).



SNMP messages are used for communication between the SNMP Manager and Agent. Key message types include:

- **GetRequest**: Retrieves data from an agent.
- **GetNextRequest**: Fetches the next object in the MIB hierarchy.
- **SetRequest**: Modifies device configuration.
- **GetBulkRequest**: Retrieves large sets of data efficiently (SNMPv2+).
- **Trap**: Unsolicited notification from the agent to the manager (e.g., error or event).
- **InformRequest**: Acknowledged notification from agent to manager (SNMPv2+).
- **Response**: Agent's reply to a request.
- **Report**: Handles error or security issues in SNMPv3.

5.3.E-MAIL

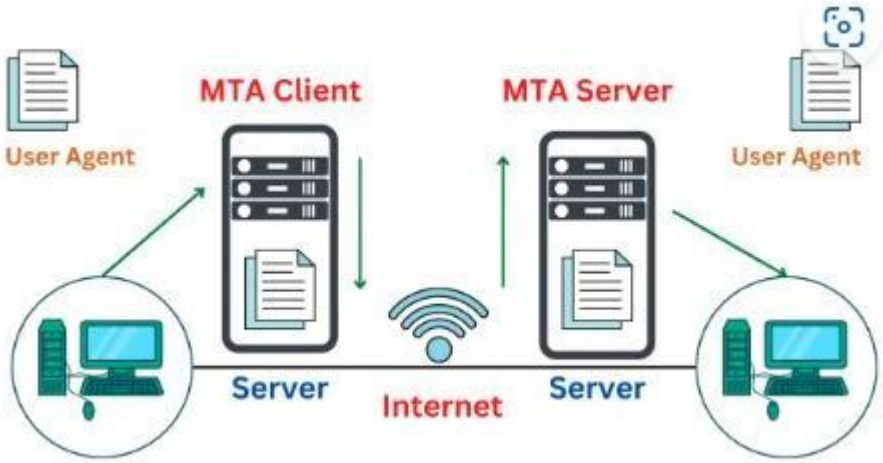
Electronic mail, commonly known as email, is a method of exchanging messages over theinternet. Here are the basics of email:

To send an email:

1. Compose a new message in your email client.
2. Enter the recipient's email address in the "To" field.
3. Add a subject line to summarize the content of the message.
4. Write the body of the message.
5. Attach any relevant files if needed.
6. Click "Send" to deliver the message to the recipient's email server.
7. Emails can also include features such as cc (carbon copy) and bcc (blind carboncopy) to send copies of the message to multiple recipients, and reply, reply all, and forward options to manage the conversation.

Electronic Mail (e-mail) is one of most widely used services of Internet. This service allows an Internet user to send a **message in formatted manner (mail)** to the other Internet user in any part of world. Message in mail not only contain text, but it also contains images, audio and videos data. The person who is sending mail is called **sender** and person who receives mail is called **recipient**. It is just like postal mail service. **Components of E-Mail System :** The basic components of an email system are : User Agent (UA), Message Transfer Agent (MTA), Mail Box, and Spool file. These are explained as following below.

1. **User Agent (UA) :** The UA is normally a program which is used to send and receive mail. Sometimes, it is called as mail reader. It accepts variety of commands for composing, receiving and replying to messages as well as for manipulation of the mailboxes.
2. **Message Transfer Agent (MTA) :** MTA is actually responsible for transfer of mail from one system to another. To send a mail, a system must have client MTA and system MTA. It transfer mail to mailboxes of recipients if they are connected in the same machine. It delivers mail to peer MTA if destination mailbox is in another machine. The delivery from one MTA to another MTA is done



3. **Mailbox :** It is a file on local hard drive to collect mails. Delivered mails are present in this file. The user can read it delete it according to his/her requirement. To use e-mail system each user must have a mailbox . Access to mailbox is only to owner of mailbox.
4. **Spool file :** This file contains mails that are to be sent. User agent appends outgoing mails in this file using SMTP. MTA extracts pending mail from spool file for their delivery. E-mail allows one name, an **alias**, to represent several different e-mail addresses. It is known as **mailing list**, Whenever user have to sent a message, system checks recipient's name against alias database. If mailing list is present for defined alias, separate messages, one for each entry in the list, must be prepared and handed to MTA. If for defined alias, there is no such mailing list is present, name itself becomes naming address and a single message is delivered to mail transfer entity.

The email system provides essential services such as:

1. **Mail Sending/Receiving**: Using protocols like SMTP, POP3, and IMAP to send, receive, and manage messages.
2. **Storage & Retrieval**: Storing emails on the server, with POP3 downloading messages and IMAP allowing synchronized access across devices.

3. **Mail Organization**: Features like folders, labels, sorting, and archiving to manage messages.
4. **Addressing/Directory Services**: Storing contacts and providing auto-completion and global address lists for easier communication.
5. **Security**: Authentication, encryption (SSL/TLS, PGP), spam filtering, and virus scanning for secure communication.
6. **Mail Queuing/Batching**: Queuing emails for later delivery and batching large volumes of emails for efficiency.
7. **Notifications**: Read receipts and delivery notifications to confirm email status.

Message Formats in Email

Email messages consist of two primary parts: the **header** and the **body**. These elements work together to ensure that the message is correctly formatted, delivered, and understood.

1. Header

The header contains essential information about the email, such as sender, recipient,

subject, and date. Key fields in an email header include:

- **From**: The email address of the sender.
- **To**: The recipient's email address.
- **Cc (Carbon Copy)**: Recipients who will receive a copy of the email, visible to all.
- **Bcc (Blind Carbon Copy)**: Recipients who receive a copy of the email, but their addresses are hidden from others.
- **Subject**: A brief summary of the message content.
- **Date**: The timestamp when the message was sent.
- **Message-ID**: A unique identifier for the email, used for referencing and tracking.
- **Reply-To**: Specifies the email address to which replies should be sent, if different from the sender's address.

2. Body

The body contains the main content of the email. It can be in **plain text** or **HTML format**:

- **Plain Text**: Contains only text with no formatting or images. Simple and universally readable, but lacks visual appeal.
- **HTML Format**: Allows for rich text formatting (bold, italics, font colors, etc.) and embedding of images, hyperlinks, and videos. Common for marketing and professional emails.

The body can include:

- **Greeting**: An opening statement such as "Dear [Name]," or "Hello."
- **Message Content**: The main part of the email, which can be paragraphs of text, lists, or other elements.
- **Closing**: A polite sign-off like "Best regards," "Sincerely," or "Thanks."
- **Signature**: Often included at the end of the email, containing the sender's name, title, contact information, or company details.

3. Attachments

Emails can include attachments (files, images, documents, etc.). Attachments are typically added using a paperclip icon or an "Attach" button in most email clients. The attachment is encoded in the email message and typically appears at the bottom or as part of the email body.

Email Message Transfer Process :

1. **Sender Composes Email**: The sender creates an email using an email client.
2. **SMTP (Sender's Server)**: The email is sent via SMTP to the sender's mail server.
3. **DNS Lookup**: The sender's mail server queries DNS to find the recipient's mail server (MX record).
4. **MTA (Recipient's Server)**: The email is transferred from the sender's MTA to the recipient's MTA.
5. **Recipient's Mailbox**: The email is stored in the recipient's mailbox on their server.
6. **POP3/IMAP**: The recipient's email client retrieves the email using POP3 (downloads) or IMAP (syncs with the server).
7. **Recipient Reads Email**: The recipient opens and reads the email.

Final Delivery in Email Process:

Final delivery refers to the last stage of the email transmission where the email is successfully delivered to the recipient's inbox. Here's a concise breakdown of the process:

1. **Message Reaches Recipient's Mail Server**:
    - The email arrives at the recipient's **Mail Transfer Agent (MTA)** via SMTP.
    - The recipient's mail server stores the message in the recipient's **Mailbox**.
2. **Email Retrieval**:
    - The recipient's **email client** (using POP3 or IMAP) accesses the mailbox to retrieve the email.
3. **Recipient Reads the Email**:
    - The email is opened by the recipient in their email client for viewing.

Once the email is in the recipient's inbox, it's considered as successfully delivered and available for reading.

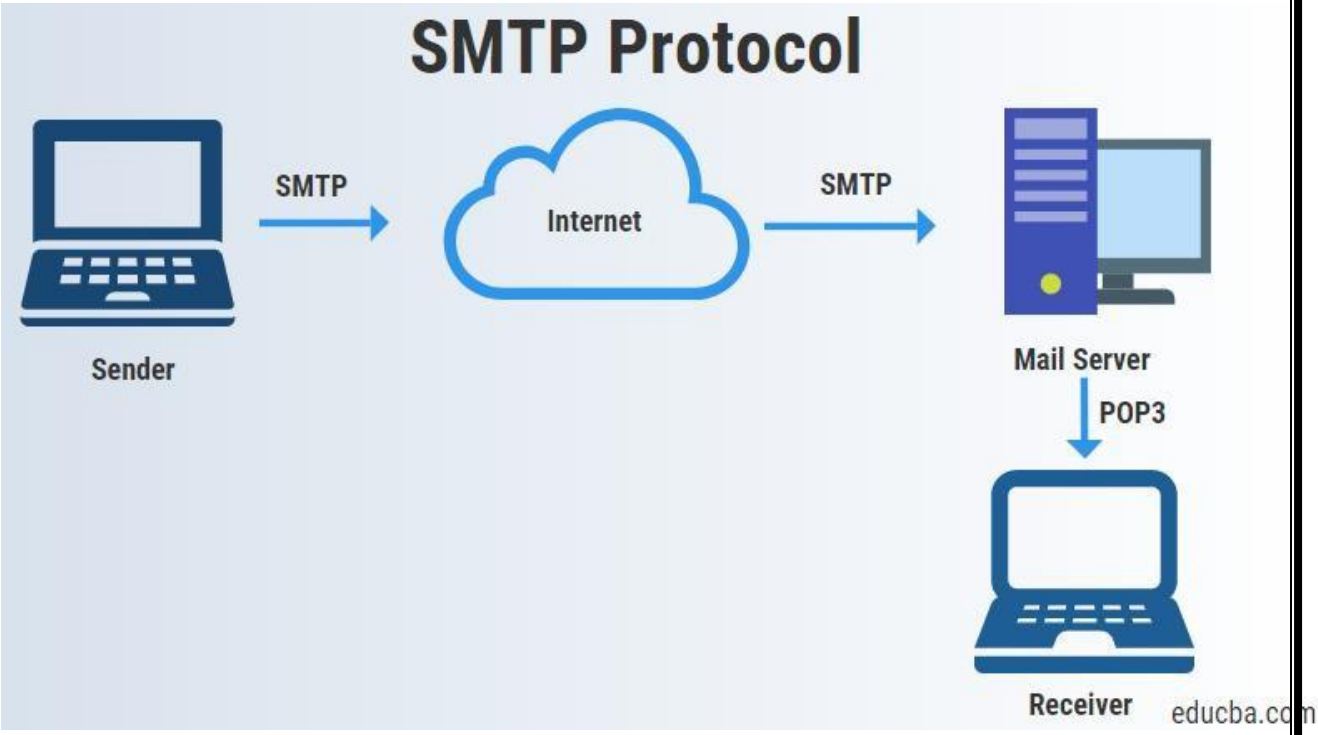
SMTP (Simple Mail Transfer Protocol)

SMTP (Simple Mail Transfer Protocol) is one of the fundamental protocols used to send emails over the internet. It is an application-layer protocol defined in **RFC 5321**, and is a text-based protocol used for transferring emails from the sender's email client to the recipient's email server, and sometimes between two email servers.

How SMTP Works:

SMTP works in conjunction with other protocols like POP3 or IMAP for retrieving messages from the server. Here's a deeper dive into how it operates:

1. **Sender's Email Client to Sender's Mail Server (SMTP Client)**:
    - When you click **send** on your email client (e.g., Gmail, Outlook), the email client sends the message to the **Mail Transfer Agent (MTA)** using SMTP.
    - The MTA at the sender's end (often called **Outgoing Mail Server**) takes the message, adds necessary headers (such as sender, recipient, subject, etc.), and establishes a connection with the recipient's email server using SMTP.
2. **Between Two Mail Servers**:
    - If the recipient's email server is on a different network, the sender's server forwards the message to the recipient's server using SMTP.
    - SMTP uses a **store-and-forward** mechanism. This means that if the recipient's server is temporarily unavailable, the sender's server will keep trying to deliver the email until it succeeds.
3. **Recipient's Mail Server**:
    - Once the recipient's server receives the message, it stores it until the recipient retrieves it using **IMAP** or **POP3** (depending on how the recipient has set up their email client).
    - SMTP is responsible for sending the email, but not for managing the messages in the inbox.



POP3 (Post Office Protocol Version 3)

POP3 is a protocol used to provide access to an email inbox stored on an email server. It allows a user to download and delete messages from the server to their local device. Once the POP3 client (email program) establishes a connection with the mail server, it retrieves all the messages stored on the server. The key feature of POP3 is that it allows users to access their emails offline, meaning they can read and manage emails without an internet connection after downloading them.

Key Features of POP3:

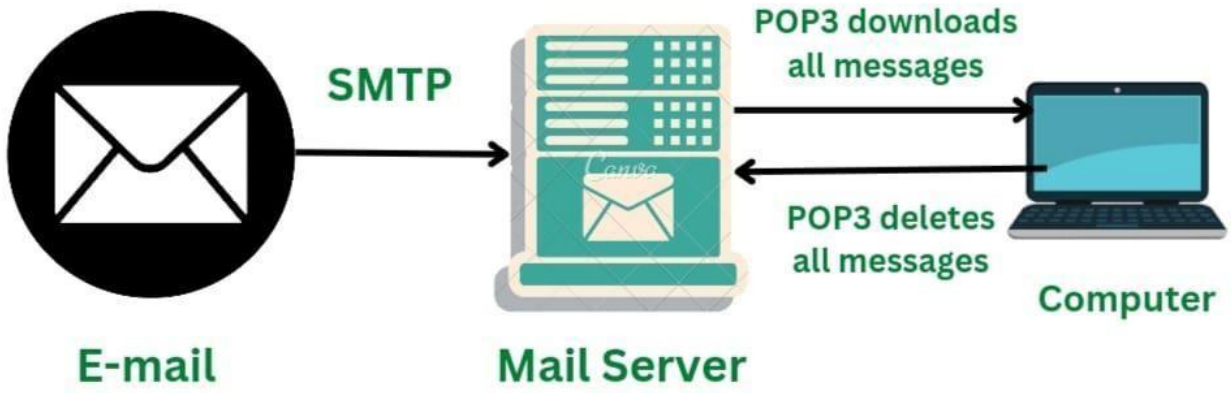
- **Offline Access**: Once emails are downloaded to the client, users can access and read them offline.
- **Download and Delete**: POP3 can download emails from the server and delete them from the server once retrieved, depending on the configuration. This can free up server storage.
- **Manual Synchronization**: POP3 does not support real-time synchronization of messages between the server and client. Users can configure their email clients to check for new messages at intervals or manually initiate the process.

Limitations of POP3:

- **No Real-Time Synchronization**: Unlike IMAP, which keeps the email client synchronized with the server (meaning actions like deleting, marking, or moving emails are reflected across devices), POP3 does not inherently support this. Any action

performed on the downloaded emails is not automatically reflected on the server or other devices.

- **Limited Server Storage**: POP3 usually removes messages from the server after they are downloaded, meaning the server stores fewer emails, but the messages are stored locally on the user's device.



Internet Control Message Protocol (ICMP)

ICMP (Internet Control Message Protocol) is a protocol used for error reporting and management queries within an IP network. It is primarily used by network devices, such as routers, to send error messages and operational information. For example, ICMP helps notify users when a requested service is unavailable or if a host or router cannot be reached.

Role of ICMP:

- **Error Reporting**: ICMP is used to report errors that occur during the transmission of data, such as unreachable destinations or network congestion. When an error occurs, ICMP sends an error message to the sender to inform them of the issue.
- **No Built-in Error Correction in IP**: Since the IP protocol lacks its own error-reporting or error-correcting mechanisms, ICMP plays a crucial role by providing feedback to the sender when issues arise during data transmission.

How ICMP Works:

- When data is being sent to a destination, it might be intercepted or encounter issues along the route (e.g., network congestion, packet loss). Without ICMP, the sender may not be aware of these issues, and the communication might fail unnoticed.
- ICMP helps by reporting these issues back to the sender, allowing them to understand why the message didn't reach its intended destination and potentially take corrective action, such as rerouting the message or troubleshooting network problems.

Common Uses of ICMP:

- **Ping**: One of the most common uses of ICMP is the **ping** command, which sends ICMP echo request messages to a destination host and waits for an echo reply to check if the host is reachable and measure network latency.
- **Traceroute**: Another tool that uses ICMP is **traceroute**, which helps identify the path and identify potential problems along the route.

File Transfer Protocol (FTP)

FTP (File Transfer Protocol) is an Internet protocol that allows the transfer of files from one computer to another over a TCP/IP network. Developed by Abhay Bhushan in 1971, FTP provides a method for accessing remote directories or folders and enables the transfer of data, software, and text files between different types of computers. In this connection, the client computer is known as the **localhost**, while the server providing the data is referred to as the **remote host**.

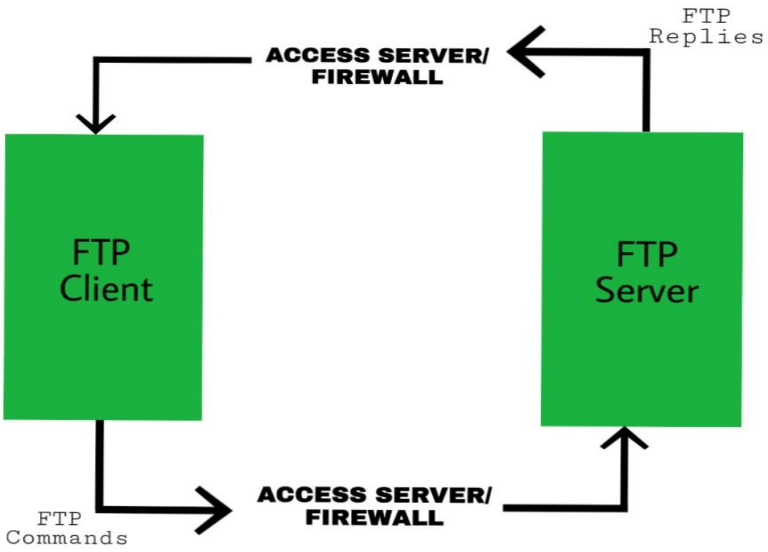
Key Features of FTP:

- **Cross-Platform File Transfer**: FTP facilitates file transfers between heterogeneous systems, meaning computers with different operating systems, directory structures, and character sets. FTP abstracts these differences and allows data to be transferred efficiently and reliably.
- **File Types Supported**: FTP supports the transfer of various file types, including:
  - **ASCII files** (text-based files), with the default encoding using **NVT ASCII**.
  - **EBCDIC files** (used by IBM mainframes).
  - **Binary image files**, which are transferred in their raw format.
- **Comparison with Other Protocols**: While there are other protocols like HTTP that also transfer files, FTP is more focused and specialized for file transfers, offering greater clarity and control for tasks like directory access, file uploads, and downloads.

Advantages of FTP:

1. **Support for Different File Types**: FTP can handle both text-based and binary files, making it versatile for various data transfer needs.

2. **Cross-System Compatibility**: FTP allows file sharing between systems with different operating systems and structures, overcoming compatibility challenges.
3. **Efficient File Transfer**: FTP ensures the reliable and efficient transfer of large files between systems.



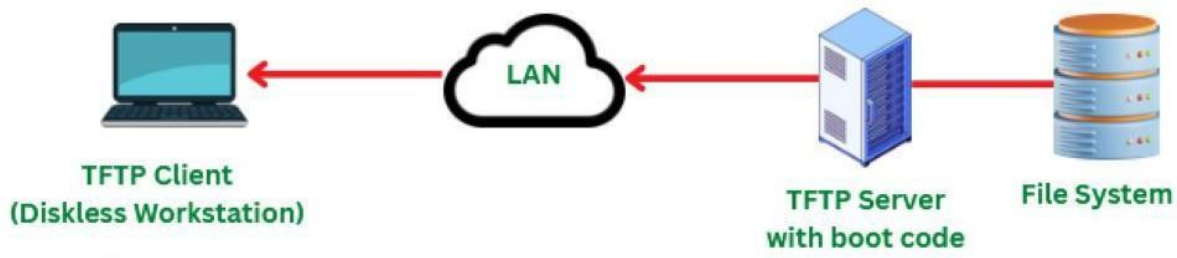
TFTP (Trivial File Transfer Protocol)

TFTP is a simple protocol used to transfer files between a client and a server, either from the client to the server or vice versa. Unlike more complex file transfer protocols, TFTP is used in situations where minimal interaction is needed between the client and server. It is designed for situations where ease of implementation and low overhead are priorities.

Key Characteristics of TFTP:

- **Protocol Type**: TFTP is a **connectionless** protocol, relying on **UDP (User Datagram Protocol)**, which does not establish a direct connection between the client and server.
- **Port Number**: TFTP operates on **UDP port 69**.
- **Security**: TFTP does **not provide security features**, which means it is unsuitable for communication over the internet where security is critical. It is mainly used in closed, trusted networks or for **local area network (LAN)** file transfers.

- **Minimal Resource Usage**: TFTP requires less memory and processing power compared to more sophisticated protocols like FTP, making it suitable for devices with limited resources, such as network devices or embedded systems.
- **Simpler Design**: Unlike FTP, which supports advanced features like authentication and directory listings, TFTP only supports the basic file transfer capabilities (read and write operations).



Working of TFTP

1. **Port and Protocol**:
   - TFTP uses **UDP** (User Datagram Protocol) and operates on **port number 69**.
2. **Client Request**:
   - The client initiates the connection by sending a **Read Request (RRQ)** if it wants to retrieve a file from the server or a **Write Request (WRQ)** if it wants to send a file to the server.
3. **File Transfer**:
   - After the request is made, the server begins transferring the file in **small packets**. Each packet has a size of **512 bytes**.
4. **Acknowledgment**:
   - After each packet is sent, the server waits for an acknowledgment (ACK) from the client confirming that the packet has been received.
   - Once the acknowledgment is received, the server sends the next 512-byte packet.
5. **Continued Transfer**:
   - This process of sending packets and waiting for acknowledgment continues until the last packet is sent and acknowledged. The last packet may contain fewer than 512 bytes if the file size is not a multiple of 512.
6. **Completion**:
   - The file transfer is considered complete when the server sends the last packet and receives the acknowledgment from the client.

Telnet:

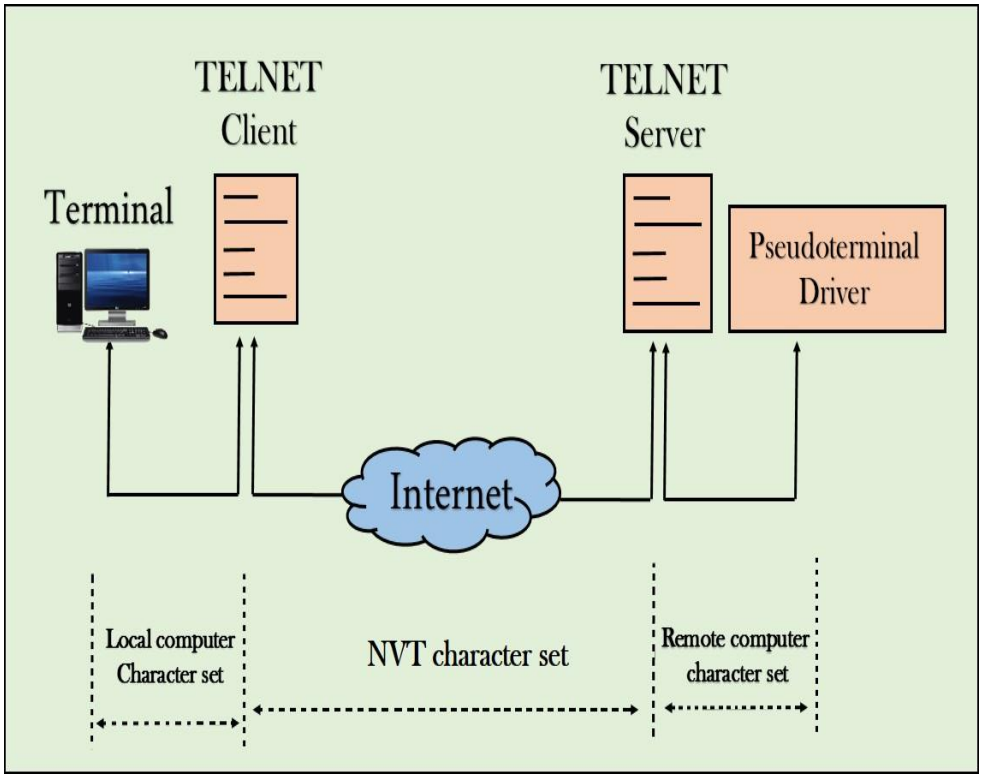
Telnet (short for **Telecommunication Network**) is one of the oldest protocols used to remotely access and manage network devices and computers. It operates on the **client-server model** and allows users to interact with a remote computer as if they were physically present in front of it. Although Telnet was widely used for remote access in the past, its security limitations have led to its decline in favor of more secure protocols like **SSH (Secure Shell)**.

How Telnet Works:

1. **Establishing a Connection**:
    - Telnet operates over **TCP/IP**, using port **23**.
    - A client initiates a connection to a remote system by providing the server's IP address and logging in with the necessary credentials (usually a username and password).
2. **Client-Server Communication**:
    - Once the client connects to the server, the server responds by presenting a command-line interface (CLI) on the client side.
    - The client sends commands to the server, and the server responds by executing the commands and displaying the output on the client's screen.
3. **Interactive Session**:
    - The user types commands on the local terminal (client), and these commands are transmitted to the remote machine for execution.
    - Output from the commands is then sent back to the client and displayed on the terminal.
4. **Remote Management**:
    - This setup allows users to manage remote devices such as network routers, servers, and other devices from anywhere, provided there is network connectivity.

Components of Telnet:

- **Telnet Client**: A software that runs on the local computer to initiate the connection to the remote host. Some operating systems (like Unix/Linux) have built-in Telnet clients, while others require the installation of third-party software.
- **Telnet Server**: A program running on the remote computer that listens for incoming Telnet connections and allows remote access to the machine's terminal interface.


TELNET
Client
TELNET
Server
Terminal
Pseudoterminal
Driver
Internet
Local computer
Character set
NVT character set
Remote computer
character set